

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

frontend/src/service/diario.service.js

```
1.  /**
2.   * @file Servicio para las operaciones del diario.
3.   * @description Proporciona métodos para interactuar con la API del
4.   * @requires ../config/api.service
5.   */
6.  import api from '../config/api.service';
7.
8.  /**
9.   * @namespace diarioService
10.  * @description Un objeto que agrupa todos los métodos de servicio p
11.  */
12.  export const diarioService = {
13.    /**
14.     * @function create
15.     * @description Crea una nueva entrada en el diario.
16.     * @param {object} diarioData - Los datos de la entrada a crear
17.     * @returns {Promise<object>} La nueva entrada creada.
18.     * @async
19.     */
20.    create: async (diarioData) => {
21.      const { data } = await api.post('/api/diario', diarioData);
22.      return data;
23.    },
24.
25.    /**
26.     * @function getAll
27.     * @description Obtiene todas las entradas del diario del usuari
28.     * @returns {Promise<Array<object>>} Una lista de las entradas c
29.     * @async
30.     */
31.    getAll: async () => {
32.      const { data } = await api.get('/api/diario');
33.      return data;
34.    },
35.
36.    /**
37.     * @function getById
38.     * @description Obtiene una entrada del diario por su ID.
39.     * @param {string} id - El ID de la entrada a obtener.
40.     * @returns {Promise<object>} La entrada del diario solicitada.
41.     * @async
42.     */
43.    getById: async (id) => {
44.      const { data } = await api.get(`/api/diario/${id}`);
45.      return data;
46.    },
47.
48.    /**
49.     * @function update
50.     * @description Actualiza una entrada del diario existente.
51.     * @param {string} id - El ID de la entrada a actualizar.
52.     * @param {object} diarioData - Los nuevos datos para la entrada
```

```

53.      * @returns {Promise<object>} La entrada actualizada.
54.      * @async
55.      */
56.      update: async (id, diarioData) => {
57.          const { data } = await api.put(`/api/diario/${id}`, diarioData);
58.          return data;
59.      },
60.
61.      /**
62.       * @function delete
63.       * @description Elimina una entrada del diario.
64.       * @param {string} id - El ID de la entrada a eliminar.
65.       * @returns {Promise<object>} Un mensaje de confirmación.
66.       * @async
67.       */
68.      delete: async (id) => {
69.          const { data } = await api.delete(`/api/diario/${id}`);
70.          return data;
71.      },
72.
73.      /**
74.       * @function accessWithPassword
75.       * @description Accede a una entrada de diario protegida mediante contraseña.
76.       * @param {string} id - El ID de la entrada a la que se quiere acceder.
77.       * @param {string} password - La contraseña para acceder a la entrada.
78.       * @returns {Promise<object>} La entrada del diario si la contraseña es correcta.
79.       * @async
80.       */
81.      accessWithPassword: async (id, password) => {
82.          const { data } = await api.post(`/api/diario/${id}/acceso`, { password });
83.          return data;
84.      }
85.  };

```

Documentation generated by [JSDoc 4.0.5](#) on Fri Feb 13 2026 08:42:37 GMT+0000 (Coordinated Universal Time) using the [docdash](#) theme.